

# EDS Lab Assignments

## Practical 5

**Name:** Sudipt Kale

**PRN:** 202501040097

### 5.1.1 Stacked Plot

#### Problem Statement:

Create a stacked area plot to visualize the temperature variations for three different cities (City A, City B, and City C) across the months of the year. The temperature data is provided for each city in the editor.

Your task is to:

- Create a stacked area plot using the data.
- Label the x-axis as "Month", the y-axis as "Temperature", and provide the title "Temperature Variation" for the plot.
- Display the plot showing the temperature variation for each city throughout the months of the year.

#### Code:

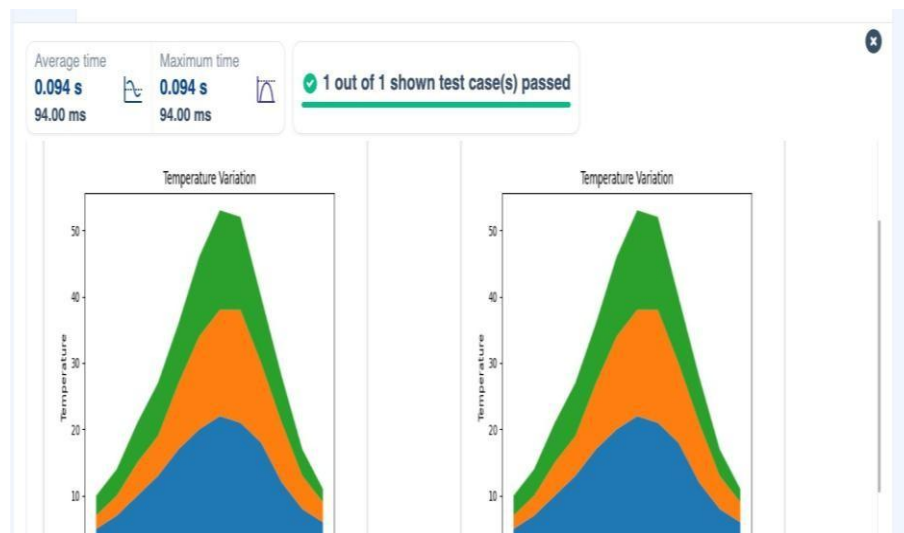
```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities data
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
              'September', 'October', 'November', 'December'],
    'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
    'City_B_Temperature': [2, 3, 5, 6, 10, 14, 16, 17, 12, 9, 5, 3],
    'City_C_Temperature': [3, 4, 6, 8, 9, 12, 15, 14, 10, 7, 4, 2]
}

# Write your code...
df = pd.DataFrame(data)

plt.stackplot(df['Month'], df['City_A_Temperature'],
              df['City_B_Temperature'], df['City_C_Temperature'])
plt.xlabel('Month')
plt.ylabel('Temperature')
plt.title('Temperature Variation')
plt.legend(loc='upper left')
plt.show()
```

#### Output:



### 5.2.1 Titanic Dataset

#### Problem Statement:

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

#### Dataset Information:

The dataset is stored in a CSV file named `titanic.csv` and has been loaded using the `pandas` library. It contains the following columns:

- Pclass: Passenger class (1 = First, 2 = Second, 3 = Third).
  - Gender: Gender of the passenger (male /female).
  - Age: Age of the passenger.
  - Survived: Survival status (0 = Did not survive, 1 = Survived). Fare:
- Ticket fare paid by the passenger.

#### Visualization:

To represent these trends, you will create 5 visualizations using Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

#### Visualization Details:

Write the code to create a series of visualizations as follows:

##### Bar Plot (Pclass Distribution):

- Create a bar plot to show the distribution of passengers across the different passenger classes (Pclass).
- Use the color skyblue for the bars.
- Title the plot as "Passenger Class Distribution".
- Label the x-axis as "Pclass" and the y-axis as "Count".

##### Pie Chart (Gender Distribution):

- Create a pie chart to display the distribution of male and female passengers.
- Use lightblue for males and lightcoral for females.
- Include percentages on the slices (use autopct='% 1.1f%%').
- Title the plot as "Gender Distribution".

##### Histogram (Age Distribution):

- Create a histogram to visualize the distribution of passengers' ages.
- Use lightgreen for the bars with black edges (edgecolor = 'black').
- Set the number of bins to 8 for the histogram.
- Title the plot as "Age Distribution".
- Label the x-axis as "Age" and the y-axis as "Frequency".

##### Bar Plot (Survival Count):

- Create a bar plot to show the count of passengers who survived and those who did not, based on the Survived column.
- Use the colors lightblue for survivors (1) and lightcoral for non-survivors (0).
- Title the plot as "Survival Count".
- Label the x-axis as "Survived (0 = No, 1 = Yes)" and the y-axis as "Count".

##### Scatter Plot (Fare vs Age):

- Create a scatter plot to visualize the relationship between the Fare and Age of passengers.
- Use orange for the data points.
- Title the plot as "Fare vs Age".
- Label the x-axis as "Age" and the y-axis as "Fare".

#### Code:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset from the CSV file df
df = pd.read_csv('titanic.csv')
```

```

# Set up the figure for 5 subplots fig, axes =
plt.subplots(3, 2, figsize=(12, 12))

# Plot 1: Count of passengers by class
axes[0, 0].bar(df['Pclass'].value_counts().index, df['Pclass'].value_counts(),
              color='skyblue')
axes[0, 0].set_title("Passenger Class Distribution")
axes[0, 0].set_xlabel("Pclass") axes[0, 0].set_ylabel("Count")

# Plot 2: Gender distribution
axes[0, 1].pie(df['Gender'].value_counts(), labels=df['Gender'].value_counts().index,
              autopct='% 1.1f%% ', colors=['lightblue', 'lightcoral'])
axes[0, 1].set_title("Gender Distribution")

# Plot 3: Age distribution
axes[1, 0].hist(df['Age'].dropna(), bins=8, color='lightgreen', edgecolor='black')
axes[1, 0].set_title("Age Distribution") axes[1, 0].set_xlabel("Age") axes[1, 0].set_ylabel("Frequency")

# Plot 4: Survival count
axes[1, 1].bar(df['Survived'].value_counts().index, df['Survived'].value_counts(),
              color=['lightblue', 'lightcoral'])
axes[1, 1].set_title("Survival Count")
axes[1, 1].set_xlabel("Survived (0 = No, 1 = Yes)") axes[1, 1].set_ylabel("Count")

# Plot 5: Fare vs Age
axes[2, 0].scatter(df['Age'], df['Fare'], color='orange', edgecolors='black')
axes[2, 0].set_title("Fare vs Age") axes[2, 0].set_xlabel("Age") axes[2, 0].set_ylabel("Fare")

plt.tight_layout()
plt.show()

```

**Output:**

Average time  
**1.240 s**  
1240.00 ms



Maximum time  
**1.240 s**  
1240.00 ms



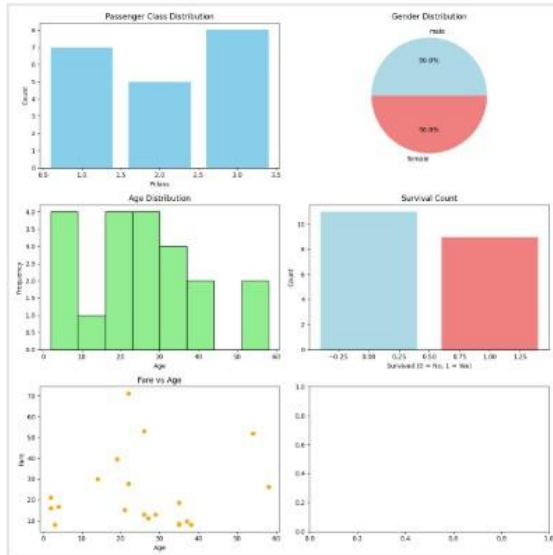
✓ 1 out of 1 shown test case(s) passed

✓ Test case 1 1240 ms

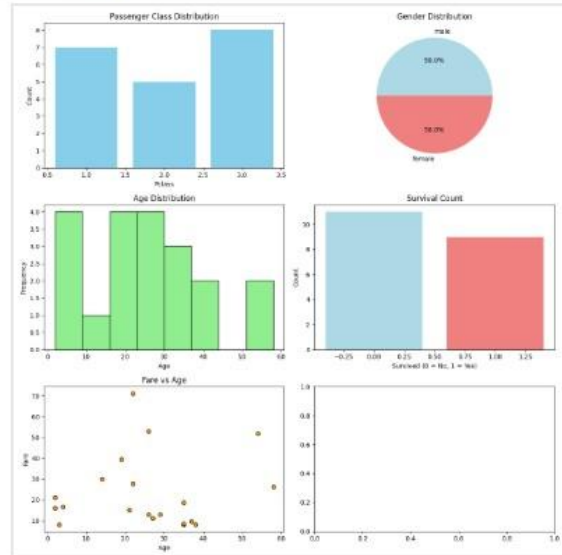
Debug



Expected output



Actual output



## 5.2.2 Histogram of Passenger Information of Titanic

### Problem Statement:

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications:

1. Use **30 bins** for the histogram.
2. Set the **edge color** of the bars to **black** (k).
3. Label the x-axis as **'Age'** and the y-axis as **'Frequency'**.
4. Add the title **"Age Distribution"** to the histogram.
5. Set the **edge color** of the bars to **black** (k).
6. Label the x-axis as **'Age'** and the y-axis as **'Frequency'**.
7. Add the title **"Age Distribution"** to the histogram.
8. Set the **edge color** of the bars to **black** (k).
9. Label the x-axis as **'Age'** and the y-axis as **'Frequency'**.
10. Add the title **"Age Distribution"** to the histogram. **Code:**

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex'] =
data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)

# Write your code here for Histogram
plt.hist(data['Age'], bins = 30, edgecolor =
'k') plt.title("Age Distribution")
plt.xlabel('Age') plt.ylabel("Frequency")
plt.show()
```

### Output:

Average time

**0.356 s**  
356.00 ms



Maximum time

**0.356 s**  
356.00 ms



✓ 1 out of 1 shown test case(s) passed

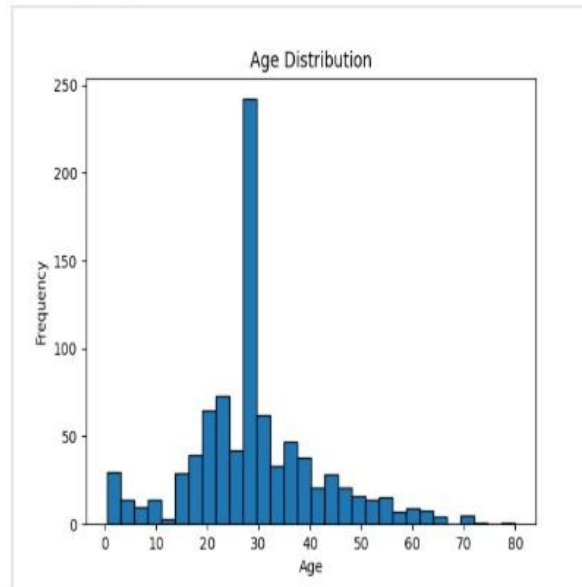
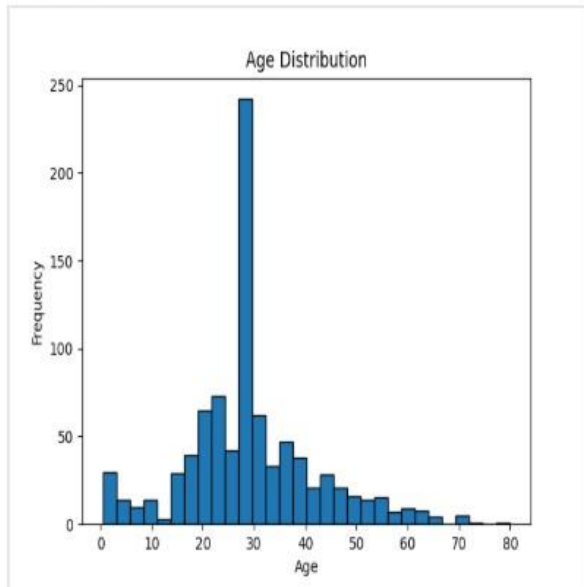
✓ Test case 1 356 ms

Debug



Expected output

Actual output



### 5.2.3 Bar Plot of Survival Rate of Passengers

#### Problem Statement:

Write a Python code to plot a bar chart that shows the count of passengers who survived and did not survive in the Titanic dataset. The chart should display the following specifications:

1. Use the '**Survived**' column to show the count of survivors (0 = Did not survive, 1 = Survived).
2. Set the chart type to '**bar**'.
3. Add the title "**Survival Count**" to the chart.
4. Label the x-axis as '**Survived**' and the y-axis as '**Count**'.

#### Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
```

```

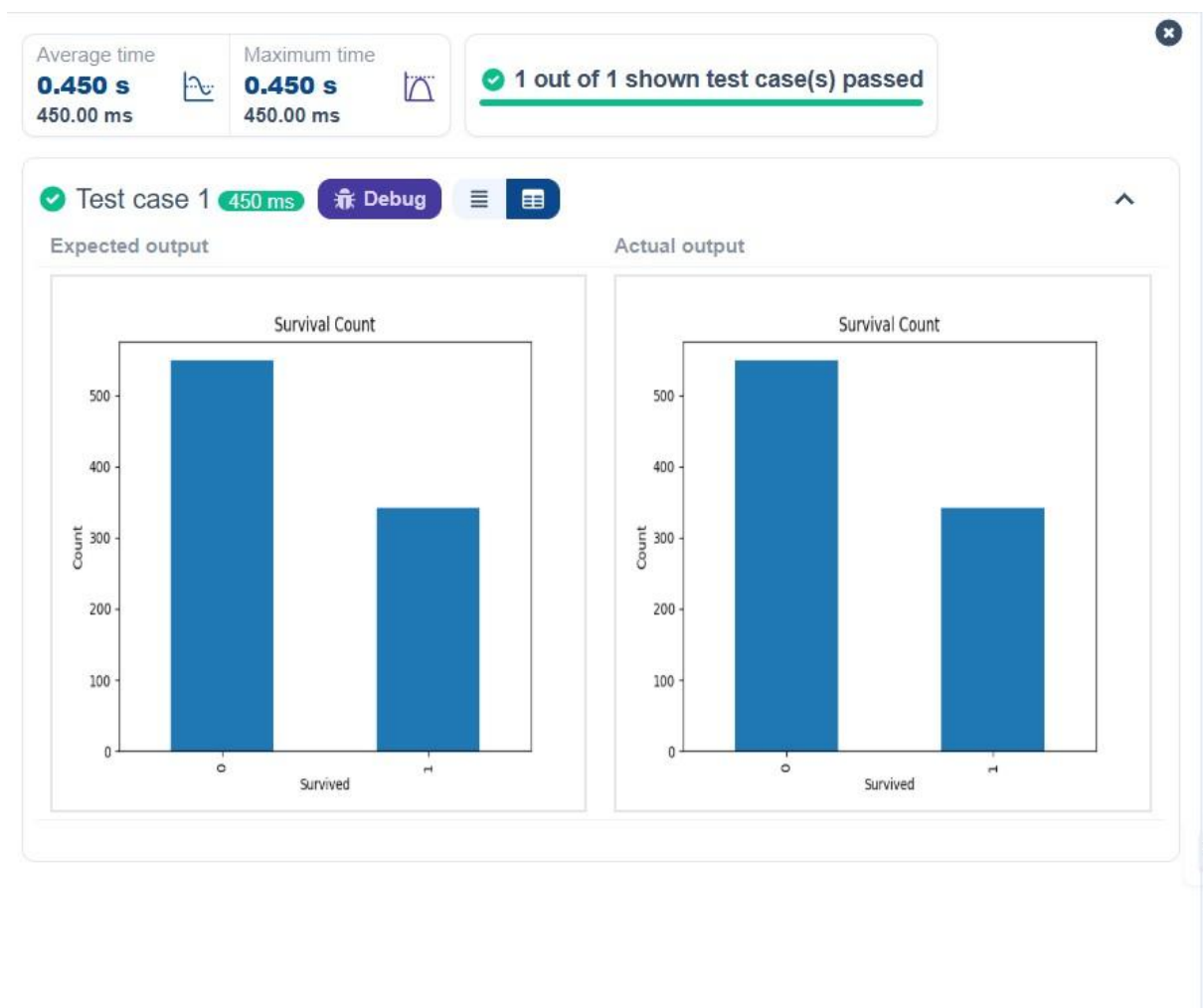
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex'] =
data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival Rate
survival_counts = data['Survived'].value_counts()
survival_counts.plot(kind='bar') plt.title('Survival
Count') plt.xlabel('Survived') plt.ylabel('Count')
plt.show()

```

**Output:**



## 5.2.4 Bar Plot for Survival by Gender

**Problem Statement:**

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by gender, in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **'Sex'** column, then use the **value\_counts()** function to count the occurrences of survivors (0 = Did not survive, 1 = Survived) for each gender.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Gender"** to the chart.
4. Label the x-axis as **'Gender'** and the y-axis as **'Count'**.
5. The legend should indicate **'Not Survived'** and **'Survived'**.

**Code :**

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex'] =
data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Gender
survived_counts = data.groupby('Sex')['Survived'].value_counts().unstack()
survived_counts.plot(kind='bar', stacked=True)

plt.title('Survival by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(['Not Survived', 'Survived']) plt.show()
```

**Output :**



### 5.2.5 Bar Plot for Survival by Pclass

#### Problem Statement:

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by passenger class (**Pclass**), in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **Pclass** column and count the number of survivors (0 = Did not survive, 1 = Survived) for each class using **value\_counts()**.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title "**Survival by Pclass**" to the chart.
4. Label the x-axis as '**Pclass**' and the y-axis as '**Count**'.
5. The legend should indicate '**Not Survived**' and '**Survived**'.

#### Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')
```

```

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex'] =
data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Pclass
survival_counts = data.groupby('Pclass')['Survived'].value_counts().unstack().fillna(0)

survival_counts.plot(kind='bar', stacked=True)

plt.title('Survival by Pclass')
plt.xlabel('Pclass')
plt.ylabel('Count')

plt.legend(['Not Survived', 'Survived']) plt.show()

```

**Output:**

Average time

**0.668 s**

668.00 ms



Maximum time

**0.668 s**

668.00 ms



✓ 1 out of 1 shown test case(s) passed

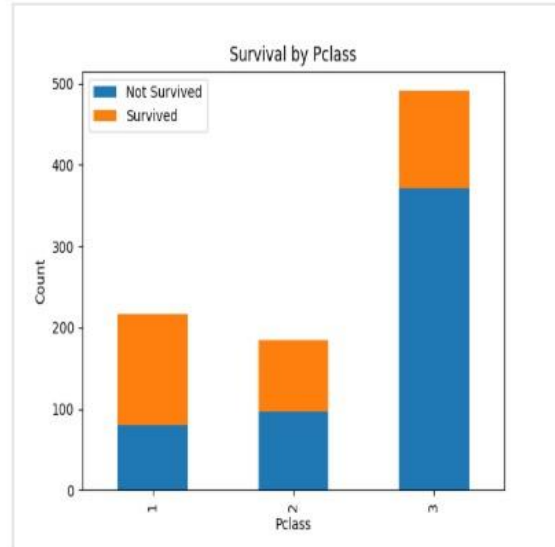
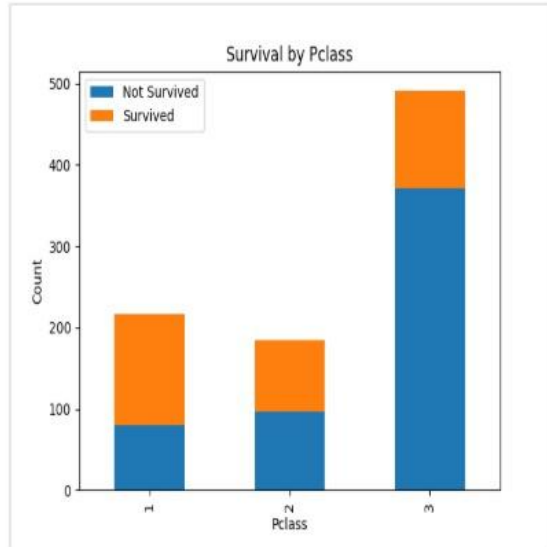
✓ Test case 1 668 ms

Debug



Expected output

Actual output



### 5.2.6 Bar Plot for Survival by Embarked

#### Problem Statement:

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset.

The chart should display the following specifications:

1. Use the **Embarked** column to determine the embarkation location. After converting this column into dummy variables (using `pd.get_dummies()`), plot the survival count based on the **Embarked\_Q** column (representing passengers who embarked from Queenstown) in relation to survival.
2. Set the chart type to 'bar' and make it stacked.
3. Add the title "**Survival by Embarked**" to the chart.
4. Label the x-axis as '**Embarked**' and the y-axis as '**Count**'.
5. Include a legend to distinguish between survivors and non-survivors (label the legend as '**Survived**' and '**Not Survived**').

#### Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

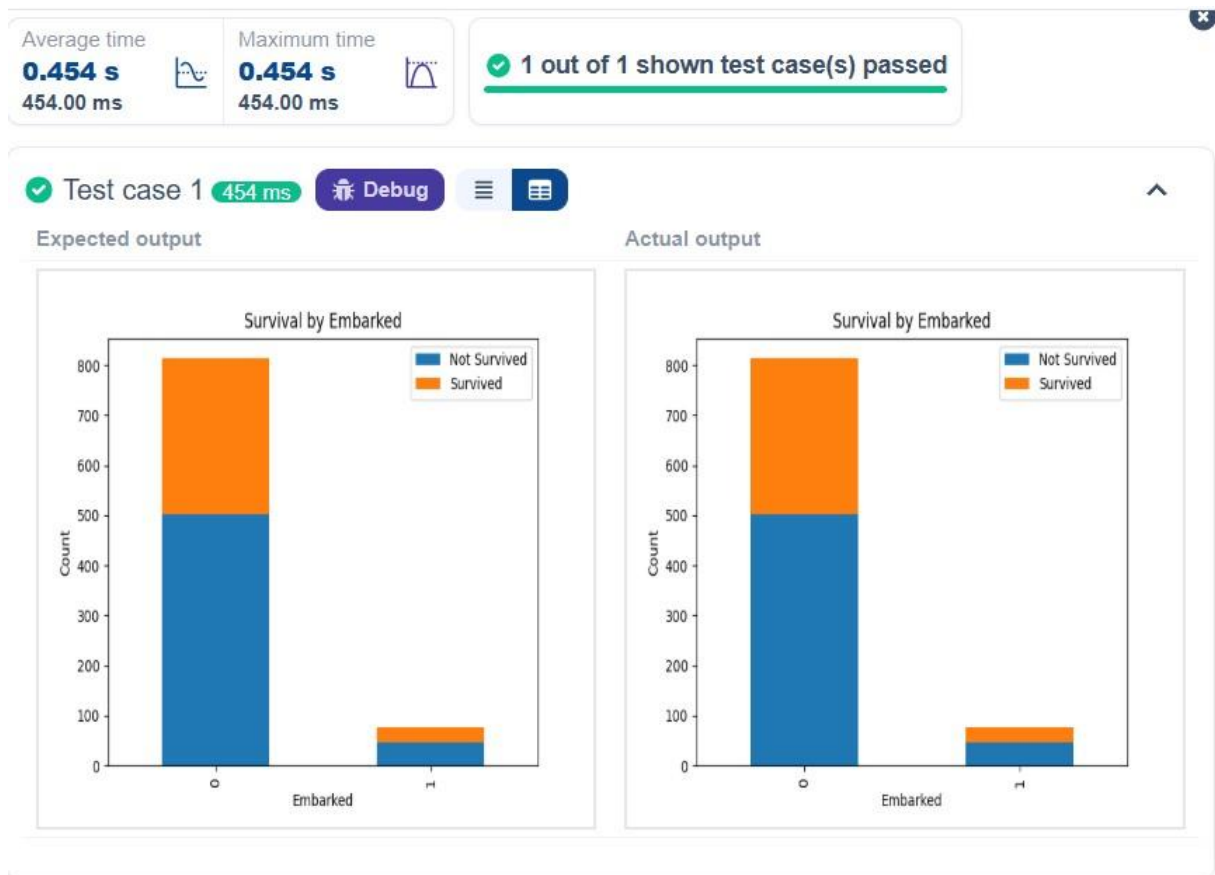
# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex'] =
data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Embarked survival_counts
= data.groupby('Embarked_Q')
[Survived].value_counts().unstack().fillna(0)

survival_counts.plot(kind='bar', stacked=True)
plt.title('Survival by Embarked')
plt.xlabel('Embarked')
plt.ylabel('Count') plt.legend(['Not
Survived', 'Survived']) plt.show()
```

Output:



### 5.2.7 Box Plot for Age Distribution

#### Problem Statement:

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle("")**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

#### Code:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex']
= data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Age by Pclass

data.boxplot(column = 'Age', by = 'Pclass')
plt.title('Age by Pclass')
plt.suptitle("")
plt.xlabel('Pclass')
plt.ylabel('Age')
plt.show()

```

Output:

Average time

0.443 s

443.00 ms

Maximum time

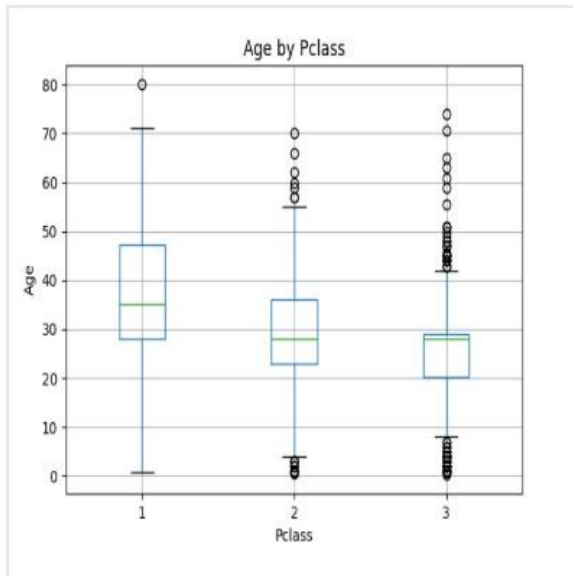
0.443 s

443.00 ms

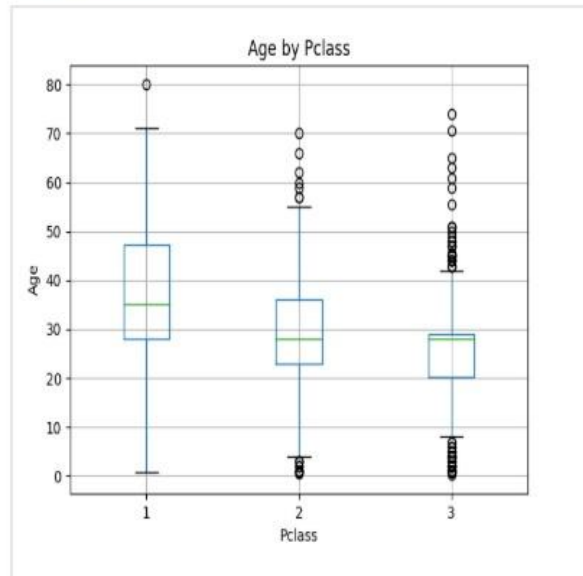
1 out of 1 shown test case(s) passed

✓ Test case 1 443 ms Debug

Expected output



Actual output



## 5.2.8 Box Plot for Age by Survived

Problem Statement:

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:

1. Use the **Survived** column to group the data for the boxplot (0 = Did not survive, 1 = Survived).
2. Set the title of the plot to **"Age by Survival"**.
3. Remove the default subtitle with **plt.suptitle("")**.
4. Label the x-axis as **'Survived'** and the y-axis as **'Age'**.

Code:

```

import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex']
= data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Age by Survived

plt.figure(figsize = (8,6))
data.boxplot(column = 'Age', by = 'Survived')
plt.title('Age by Survival')
plt.suptitle("")
plt.xlabel('Survived')
plt.ylabel('Age')
plt.show()

```

Output:

Average time  
**0.465 s**  
465.00 ms



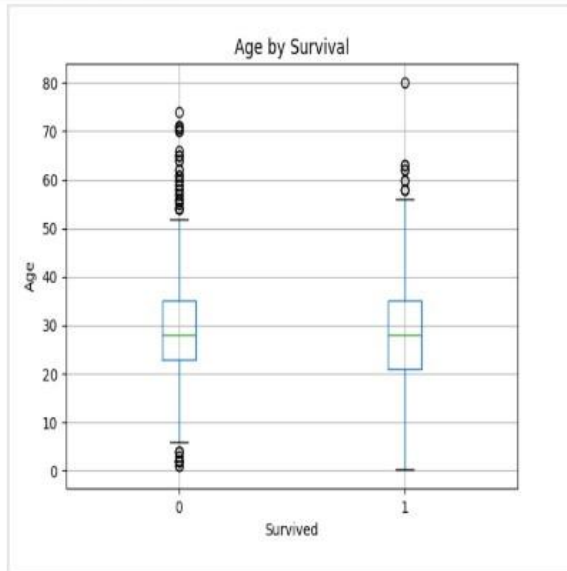
Maximum time  
**0.465 s**  
465.00 ms



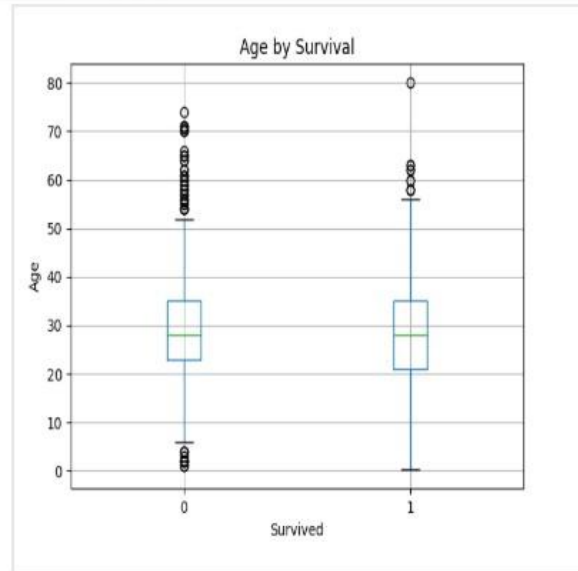
✓ 1 out of 1 shown test case(s) passed

✓ Test case 1 465 ms  Debug  

Expected output



Actual output



### 5.2.9 Box Plot for Fare by Pclass

#### Problem Statement:

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Fare by Pclass"**.
3. Remove the default subtitle with **plt.suptitle("")**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Fare'**.

#### Code:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

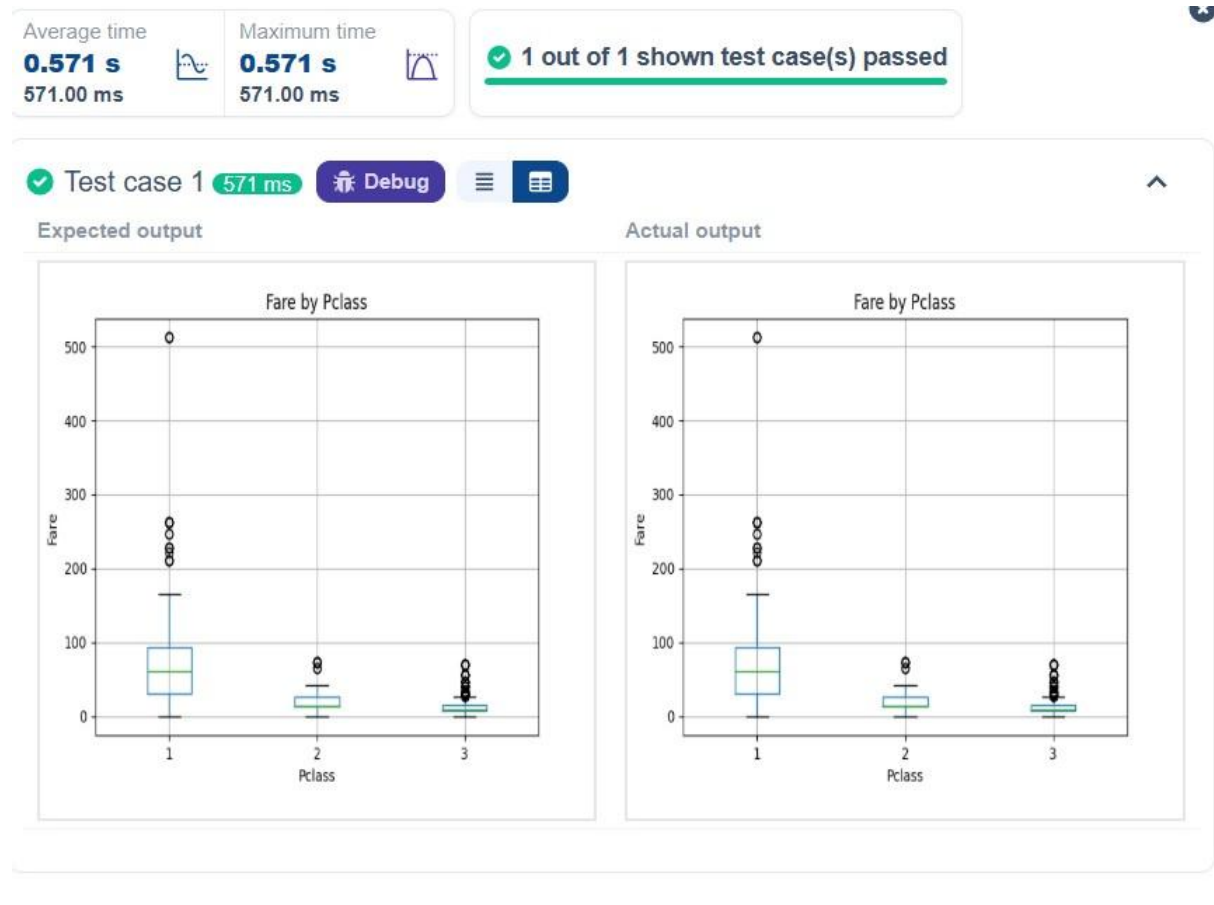
# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex']
= data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Fare by Pclass

plt.figure(figsize=(8,6))
data.boxplot(column='Fare', by='Pclass')
plt.title('Fare by Pclass')
plt.suptitle("")
plt.xlabel('Pclass')
plt.ylabel('Fare')
plt.show()
```

Output:



### 5.2.10 Scatter Plot for Age vs. Fare

Problem Statement:

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Set the title of the plot to "**Age vs. Fare**".
3. Label the x-axis as '**Age**' and the y-axis as '**Fare**'.

Code:

```

import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex']
= data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Fare by Pclass

plt.figure()
plt.scatter(data['Age'], data['Fare'])
plt.title('Age vs. Fare')
plt.xlabel('Age') plt.ylabel('Fare')
plt.show()

```

Output:



### 5.2.11 Scatter Plot for Age vs. Fare by Survived

#### Problem Statement:

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status.

The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Color the points based on the **Survived** column: **Red** for passengers who did not survive (**Survived = 0**). **Blue** for passengers who survived (**Survived = 1**).
3. Set the title of the plot to **"Age vs. Fare by Survival"**.

4. Label the x-axis as 'Age' and the y-axis as 'Fare'.

**Code:**

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset data =
pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric data['Sex']
= data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True) #

Write your code here for Scatter Plot for Age vs. Fare by Survived

colors = data['Survived'].map({0:'red', 1: 'blue'}) plt.scatter(data['Age'],
data['Fare'], c=colors)

plt.title("Age vs. Fare by
Survival") plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

**Output:**

Average time

**0.361 s**  
361.00 ms



Maximum time

**0.361 s**  
361.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ Test case 1 361 ms

Debug



Expected output

Actual output

